Project - High Level Design

on

AI-Driven
Construction Documentation System

Course Name: GenAI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vivek Sharma	EN22ME304116
2.	Jayesh Chouhan	EN23MEEL4001
3.	Sakina Modi	EN22EL301044
4.	Vinay Birla	EN22CS3011083
5.	Dev Singh Ghuraiya	EN22ME304022

Group Name:05D10

Project Number:DO-18

Industry Mentor Name:

University Mentor Name: Divya Kumawat

Academic Year:2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects
8. References

1. Introduction

1.1 Scope of the Document

This **High Level Design (HLD)** document describes the **architectural** and **design decisions** for **ConstructionGen**, an **AI-powered construction documentation platform**. It covers the overall **system architecture**, **component interactions**, **data design**, **interface definitions**, **state management strategy**, and **non-functional requirements**.

This document does not cover **low-level implementation details** such as **individual function logic**, **CSS specifics**, or **line-by-line code explanations** — those are addressed in the accompanying **LLD** and **GENAI-DOCUMENTATION** files. This document is intended to provide a **bird's-eye view** of how the system is **structured**.

In scope:

- Overall system architecture (frontend, backend, AI layer, storage)
- Component responsibilities and interactions
- API contract between frontend and backend
- Data models and storage strategy
- Session and state management approach
- Security and performance considerations

Out of scope:

- Deployment infrastructure and DevOps pipelines
- User authentication and multi-user support (not implemented in current version)
- Mobile native applications
- Third-party integrations beyond **Groq API** and **HuggingFace embeddings**

1.2 Intended Audience

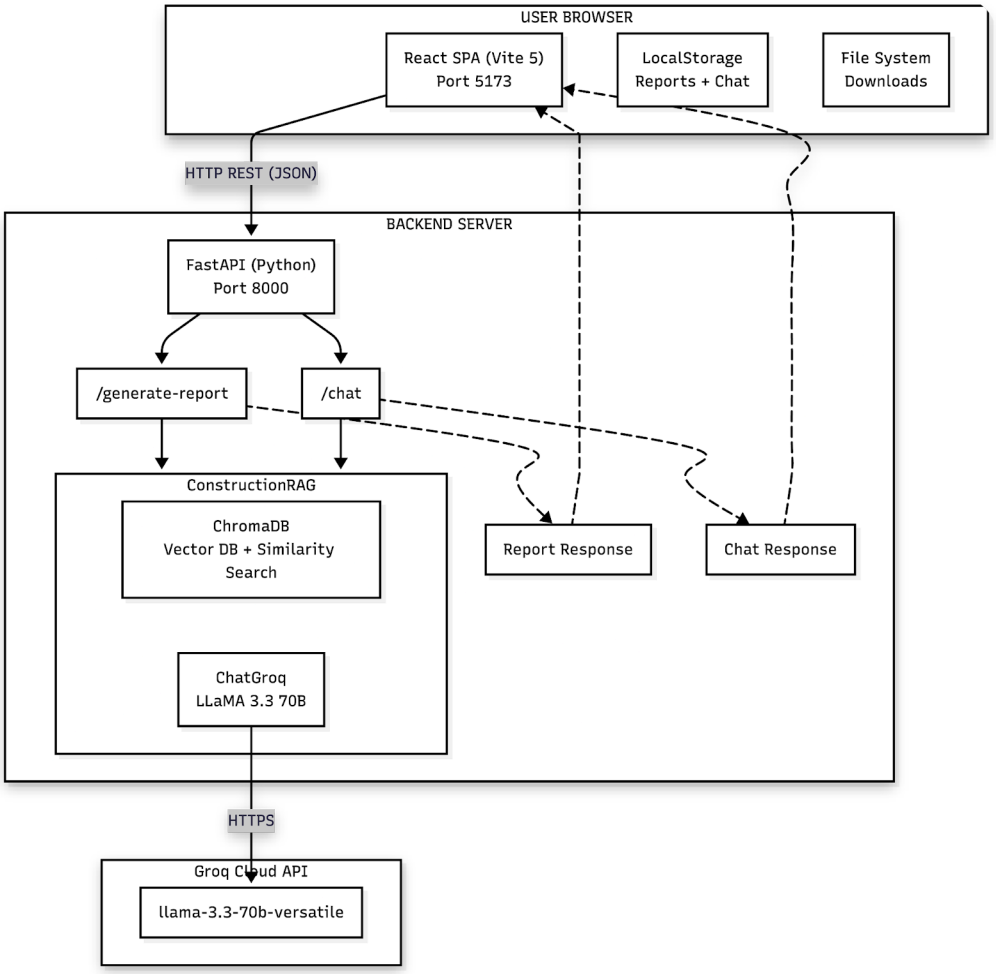
- **Software Developers**
  → Understanding system structure before contributing or extending
- **Technical Reviewers**
  → Evaluating architectural decisions and design quality
- **Project Evaluators / Assessors**
  → Reviewing the GenAI system design for academic or professional assessment
- **QA Engineers**
  → Understanding component boundaries for test planning
- **Future Maintainers**
  → Onboarding to the codebase with a high-level map

1.3 System Overview

ConstructionGen is a full-stack web application that uses Generative AI to assist construction professionals with two primary tasks:

1. **Report Generation** — Users fill a structured form with project details and a site activity description. The system uses a Retrieval-Augmented Generation (RAG) pipeline to produce a professional, formatted Daily Site Report in markdown, downloadable as PDF, Word (.docx), or plain text.
2. **AI Chat Assistant** — Users converse with an AI assistant specializing in construction knowledge. The assistant maintains full conversation memory within a session and is grounded in a local vector knowledge base to prevent hallucination.

High-Level Architecture:



Technology Summary:

- **Frontend:** React 18 + TypeScript, Vite 5, React Router v6
- **Backend:** Python FastAPI, Uvicorn ASGI server
- **AI / LLM:** Groq API, llama-3.3-70b-versatile, temperature 0.1
- **RAG / Embeddings:** LangChain, HuggingFace all-MiniLM-L6-v2
- **Vector Store:** ChromaDB (local persistent, on-disk)
- **Client Storage:** Browser localStorage
- **Styling:** Custom CSS (no Tailwind, no UI libraries)

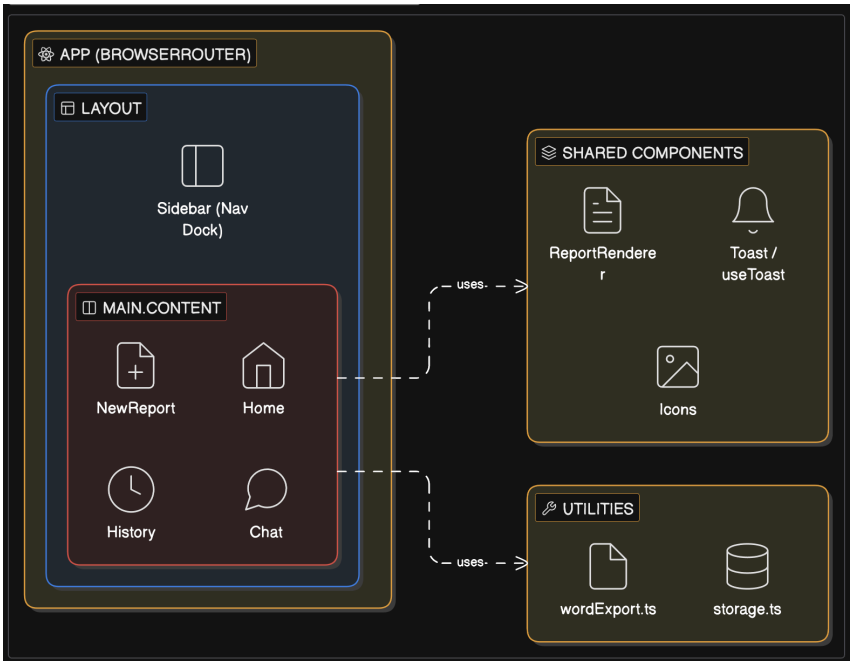
2. System Design

2.1 Application Design

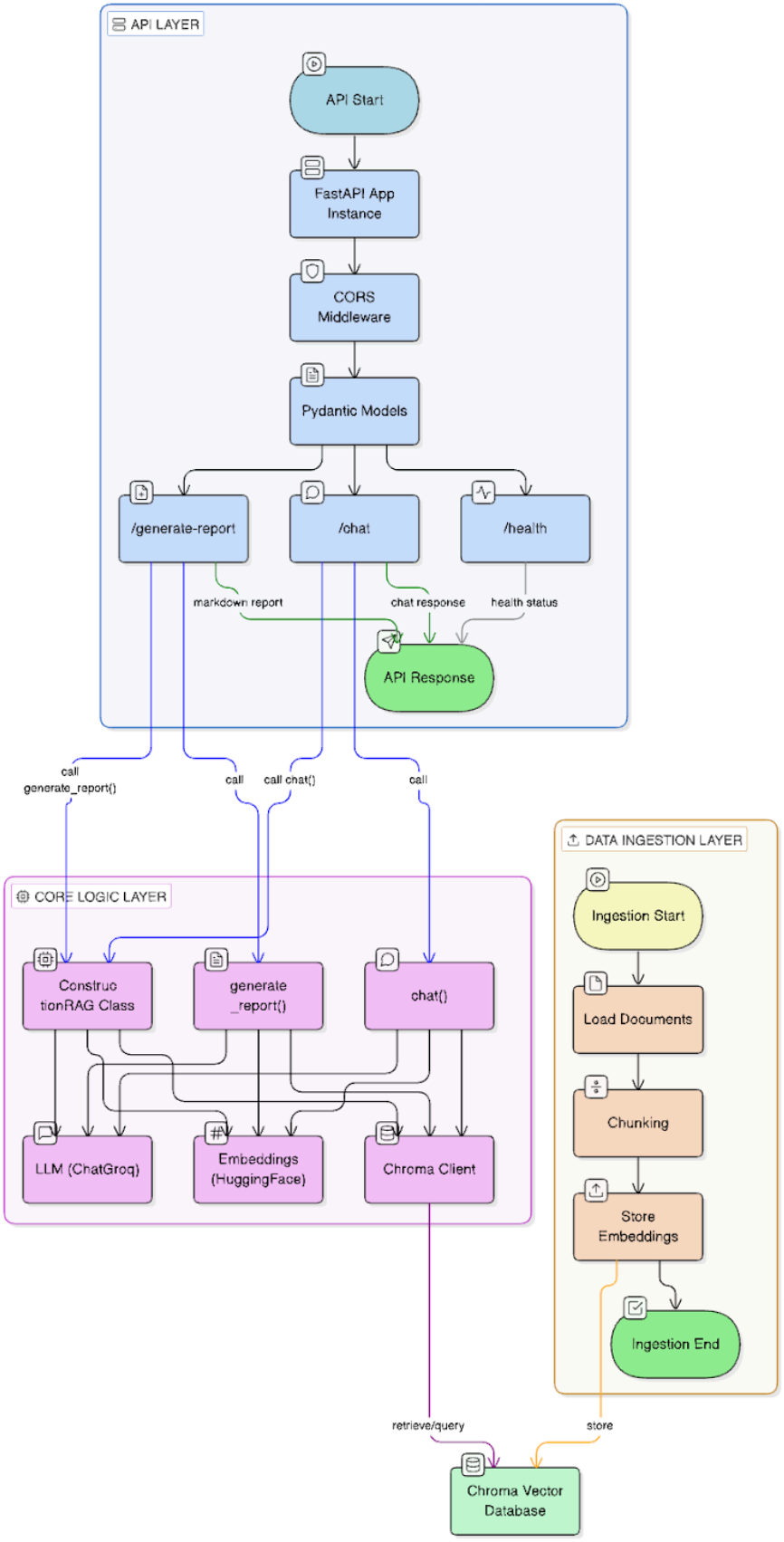
ConstructionGen follows a **two-tier client-server architecture** with an **AI inference layer**:

- **Tier 1 — Frontend (Client):** A **React Single Page Application (SPA)** running in the browser. Handles all UI rendering, form management, local data persistence, and file downloads. Communicates with the backend exclusively via **HTTP REST calls**.
- **Tier 2 — Backend (Server):** A **Python FastAPI** application. **Stateless** — holds no session data between requests. Owns the **AI pipeline**: vector retrieval, prompt construction, and LLM invocation.
- **External AI Layer: Groq Cloud API** provides **LLM inference**. The backend calls Groq synchronously per request.

Frontend Architecture — Component Tree:



Backend Architecture — Module Responsibilities:



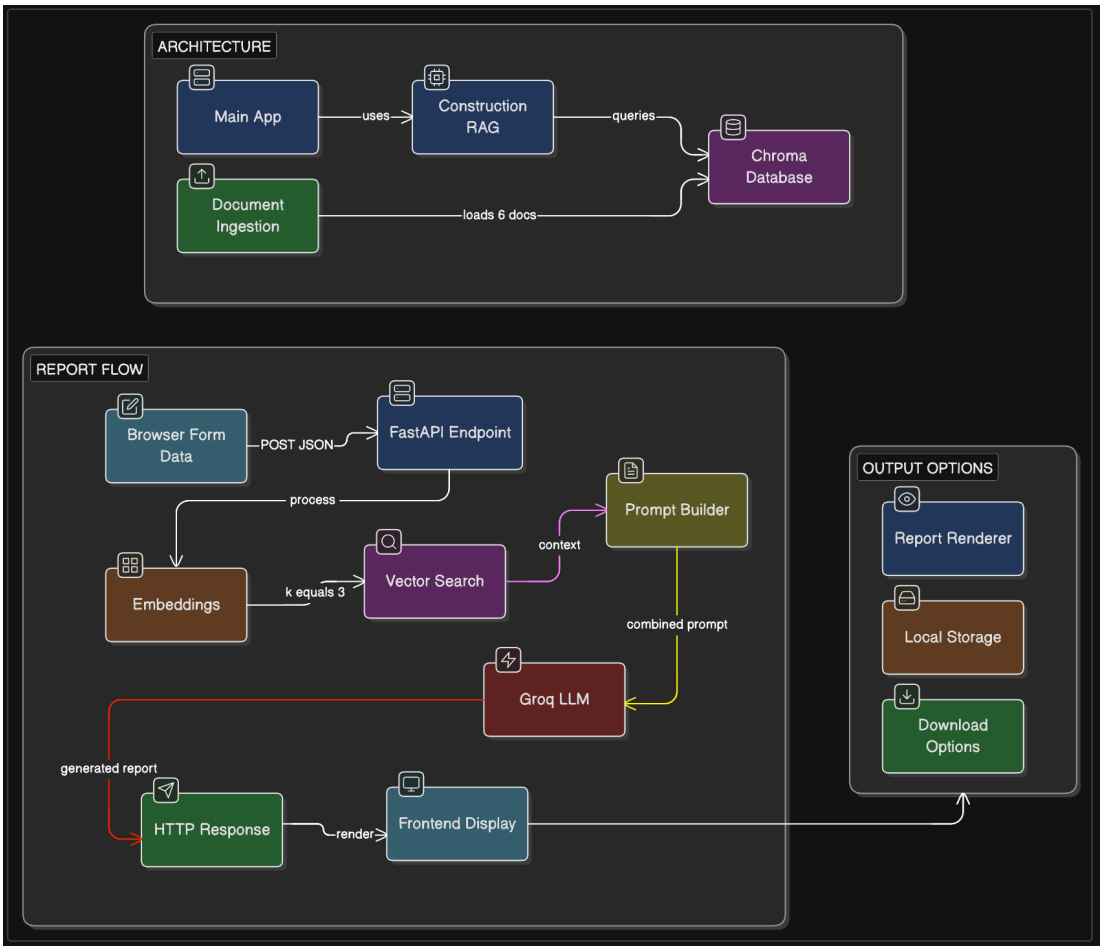
2.2 Process Flow

Report Generation Process

1. User opens `/new-report`
2. User fills form (project details, tone, length, site activity description)
3. User clicks "Generate Report"
4. Frontend validates input (topic not empty)
5. Frontend shows 6-stage progress bar
6. Frontend sends POST to `/generate-report`
7. Backend receives request
8. System generates report (embed → retrieve → prompt → LLM → markdown)
9. Backend returns `{ report, success: true }`
10. Frontend renders report
11. Report saved to localStorage
12. Toast notification shown
13. User can download (PDF / Word / TXT)

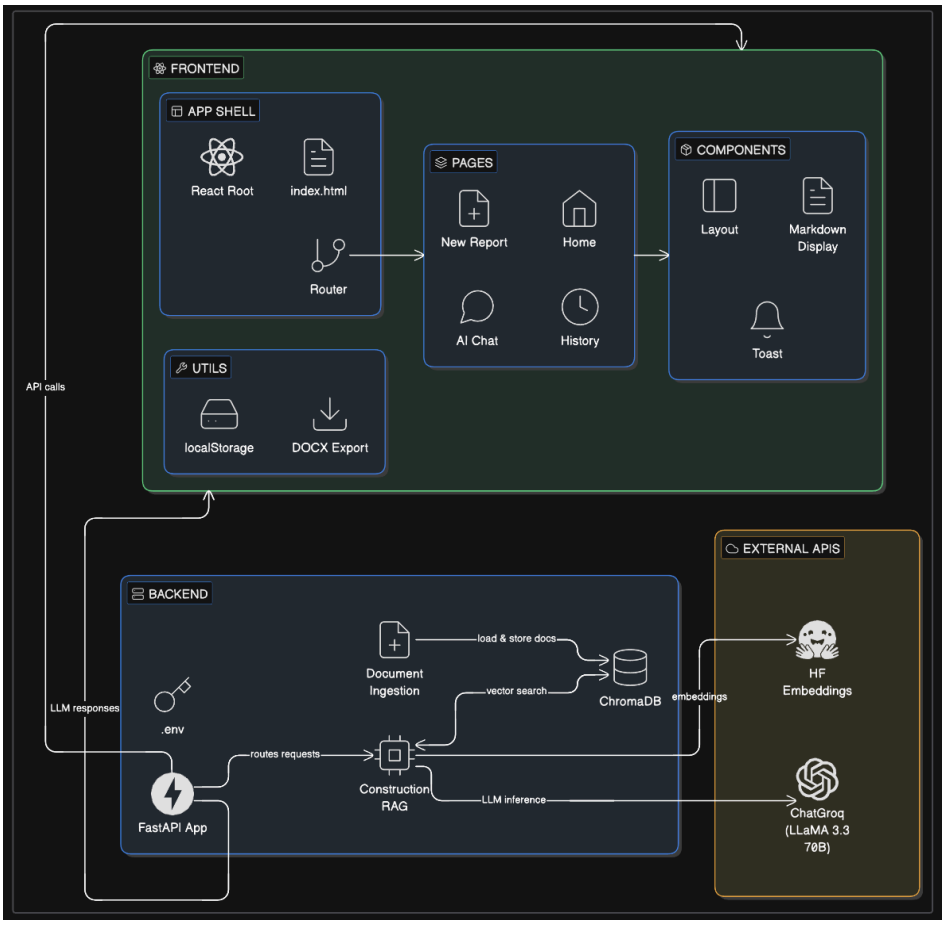
2.3 Information Flow

Report Generation — Information Flow



2.4 Components Design

The system is split into three clear layers: Frontend, Backend, and AI Pipeline. Below is the full component map followed by a breakdown of each major component.



Frontend Components

1. **App.tsx** — Router with 4 routes, wrapped in Layout
2. **Layout.tsx** — Sidebar + Outlet (CSS grid)
3. **Sidebar.tsx** — Fixed nav, active state, tooltips
4. **Home.tsx** — Hero + 3 feature cards
5. **NewReport.tsx** — Form → progress → API → render → save + download
6. **Chat.tsx** — History handling, localStorage sync, clear chat
7. **History.tsx** — Grid, search, delete, download, viewer
8. **ReportRenderer.tsx** — Markdown renderer (tables, lists, etc.)
9. **Toast.tsx / useToast** — Notifications (auto-dismiss)
10. **Icons.tsx** — SVG icon set

Backend Components

1. **main.py** — Routes, models, CORS, delegates logic
2. **ConstructionRAG (rag.py)** — Core engine (LLM + embeddings + vector DB)
3. **generate_report()** — Embed → search → prompt → LLM → markdown
4. **chat()** — Embed → search → messages → LLM → response
5. **ingest_docs.py** — Chunk + embed + store in ChromaDB

Utility Modules

1. **storage.ts** — CRUD for reports (localStorage)
2. **wordExport.ts** — Markdown → DOCX export

2.5 Key Design Considerations

1. Stateless backend — No server-side session; **history[]** sent client-side per request
2. RAG grounding — ChromaDB top-3 chunks injected per request to reduce hallucination
3. **temperature=0.1** — Near-deterministic output for factual, consistent reports
4. **localStorage** only — No backend DB for MVP; tradeoff is browser-local storage
5. Singleton **ConstructionRAG** — Avoids reloading 90MB embedding model + ChromaDB index per request
6. Markdown output — Single LLM format consumed by **react-markdown, jsPDF,** and **docx** independently
7. No Redux/Zustand — All state is page-local; cross-page data via **localStorage** utils
8. **messages[0]** excluded from history — Greeting never sent to backend; prevents model treating it as a user turn

2.6 API Catalogue

Base URL: `http://localhost:8000`

1. **GET /** — Root health check
2. **GET /health** — Service health status
3. **POST /generate-report** — Generate AI report from form data
4. **POST /chat** — Chat with AI (supports history)
5. **GET /report-types** — List available report types

3. Data Design

3.1 Data Model

ConstructionGen uses two data stores: browser localStorage for user-generated content, and ChromaDB for the AI knowledge base.

localStorage — SavedReport

```
SavedReport {
  id          : string    — Date.now().toString(), unique identifier
  projectName : string    — from form input
  projectNumber: string   — from form input
  date        : string    — ISO date string (YYYY-MM-DD)
  location    : string    — from form input
  contractor  : string    — from form input
  reportType  : string    — always "Daily Site Report" in current version
  topic       : string    — user's site activity description
  content     : string    — raw markdown string returned by AI
  createdAt   : string    — ISO timestamp of when report was saved
}
```

Stored as: `localStorage[ 'construction_reports' ]` → JSON array of `SavedReport [ ]`, newest first (prepended on save).

ChromaDB — Construction Knowledge Documents

```
Document {
  page_content : string    — text chunk (max 1000 chars)
  metadata     : {
    type        : string   — "template" | "guidelines" | "glossary" |
                             "best_practices" | "standards" | "requirements"
    report_type : string   — e.g. "daily_site_report", "safety_inspection"
    category    : string   — e.g. "terminology", "quality_control"
  }
}
```

6 source documents → split into chunks → embedded → stored in `./chroma_db` (SQLite + binary index files).

3.2 Data Access Mechanism

localStorage Access (Frontend)

1. Save report — `saveReport()` → **parse** → **prepend** → **stringify** → **setItem**
2. Load reports — `getReports()` → **getItem** → **parse** → return
3. Delete report — `deleteReport()` → **filter** → **setItem**
4. Get by ID — `getReportById()` → **find**
5. Clear all — `clearAllReports()` → set empty array
6. Chat storage — direct **getItem**/**setItem** in `Chat.tsx` with **useEffect**

ChromaDB Access (Backend)

1. Accessed via **LangChain** `Chroma` wrapper
2. `similarity_search(query, k=3)` → returns top 3 documents
3. Uses **all-MiniLM-L6-v2** for **embeddings** (384-d vector)
4. Uses **HNSW** for **nearest-neighbor search**

Groq API Access (Backend)

1. Report generation — `llm.invoke(prompt)` → string response
2. Chat (multi-turn) — `llm.invoke(messages[])` → string response
3. Output → `response.content`

3.3 Data Retention Policies

1. **Saved reports** — Stored in **localStorage**, retained indefinitely until user action, deleted via **deleteReport(id)** or **clearAllReports()**
2. **Chat messages** — Stored in **localStorage**, retained indefinitely until user action, cleared using **"Clear Chat"** (resets to greeting)
3. **ChromaDB knowledge** — Stored on local disk (**./chroma_db**), permanent (static **knowledge base**), deleted manually by removing directory
4. **API request/response** — Stored **in-memory** only, retained for request lifetime, automatically **garbage collected**
5. **LLM conversation history** — Stored **in-memory** (browser) + **localStorage**, retained for session duration, cleared via **"Clear Chat"**

3.4 Data Migration

Current version: No formal **migration mechanism** exists. The system is a **single-user**, **browser-local MVP**.

localStorage schema changes: If the `SavedReport` schema changes in a future version, existing **localStorage** data will still **parse** correctly because:

- `JSON.parse` does not fail on missing fields

- **TypeScript interfaces** use optional fields where appropriate
- New fields would default to **`undefined`** for old records

ChromaDB re-ingestion: If the **knowledge base documents** are updated, the **`chroma_db/`** directory must be deleted and **`ingest_docs.py`** re-run. There is **no incremental update mechanism** in the current version.

Future **migration path** (if **multi-user** is added):

- Replace **localStorage** with a **backend database** (**PostgreSQL** or **MongoDB**)
- Add a **migration script** to **export localStorage JSON** and **import into the database**
- Add **user authentication** (**JWT** or **OAuth**) before **database migration**

4. Interfaces

4.1 User Interface

Overview
React SPA with fixed sidebar dock and main content area, light theme only

Navigation Interface

1. Fixed vertical dock (left side, centered)
2. 4 icon-only navigation buttons with tooltips
3. Active route highlighted with indicator
4. Responsive: moves to bottom bar on mobile

Page Interfaces

1. **Home** — Hero section, 3 feature cards, 2 CTA buttons
2. **New Report** — Form, textarea, tone/length select, generate button, progress bar, report output, download actions
3. **Chat** — Message list, typing indicator, input bar, suggested questions, clear chat
4. **History** — Search, report grid, download dropdown, delete confirm, detail viewer

Notification Interface

1. Toast component (top-center, auto-dismiss in 3s)
2. Types: success, error, info
3. Used for actions like generate, copy, delete, download

Confirmation Interface

1. Full-screen blurred overlay
2. Centered dialog with animation
3. Used only for delete confirmation (no **window.confirm()**)

4.2 Backend API Interface

The frontend communicates with the backend via **HTTP REST**. All requests and responses use **Content-Type: application/json**.

Base URL: http://localhost:8000

CORS Policy:

- Allowed origins: **http://localhost:5173, http://localhost:3000**
- Allowed methods: all (*)
- Allowed headers: all (*)
- Credentials: allowed

Error Handling:

- All route handlers are wrapped in **try/except**
- Exceptions raise **HTTPException(status_code=500, detail=str(e))**
- Frontend catches fetch errors and displays inline error messages or error toast

4.3 External API Interface — Groq

- **Provider**: Groq Cloud
- **Model**: **llama-3.3-70b-versatile**
- **Authentication**: API key via GROQ_API_KEY environment variable
- **Protocol**: HTTPS REST (abstracted by LangChain ChatGroq)

Configuration:

temperature : 0.1 (near-deterministic, factual)

max_tokens : 4096 (sufficient for detailed reports)

Invocation modes:

- Single string prompt → used for report generation
- List of LangChain message objects → used for multi-turn chat

Failure handling: If Groq API is unavailable or returns an error, the exception propagates to FastAPI's exception handler and returns HTTP 500 to the frontend.

4.4 External Interface — HuggingFace Embeddings

- **Model**: sentence-transformers/all-MiniLM-L6-v2
- **Output**: 384-dimensional dense vectors
- **Loading**: Downloaded on first run, cached locally by HuggingFace
- **Usage**: Converts text to vectors for ChromaDB similarity search
- **Authentication**: Unauthenticated (public model, rate-limited without HF_TOKEN)

4.5 File Export Interfaces

1. **PDF** — jsPDF (dynamic import) → strips markdown → renders line-by-line with page breaks → `doc.save()`
2. **Word (.docx)** — docx + file-saver → parses markdown to docx structure → `Packer.toBlob()` → `saveAs()`
3. **TXT** — Native Blob API → strips markdown → creates Blob → triggers download via `<a>` click

5. State and Session Management

5.1 Frontend State Architecture

State distribution:

1. **Form field values** — NewReport → `useState<ReportData>` → Page lifetime
2. **Generated report markdown** — NewReport → `useState<string>` → Page lifetime
3. **Generation loading flag** — NewReport → `useState<boolean>` → Request lifetime
4. **Progress bar value/message** — NewReport → `useState<number/string>` → Request lifetime
5. **Inline error message** — NewReport → `useState<string>` → Page lifetime
6. **Chat messages array** — Chat → `useState<Message[]>` → Persisted (localStorage)
7. **Chat input value** — Chat → `useState<string>` → Keystroke lifetime
8. **Chat loading flag** — Chat → `useState<boolean>` → Request lifetime
9. **Suggested questions** — Chat → `useState<string[]>` → Component mount
10. **Saved reports list** — History → `useState<SavedReport[]>` → Loaded on mount (localStorage)
11. **Search term** — History → `useState<string>` → Page lifetime
12. **Selected report** — History → `useState<SavedReport | null>` → Page lifetime
13. **Download menu state** — History → `useState<string | null>` → Click lifetime
14. **Delete confirmation state** — History → `useState<string | null>` → Click lifetime
15. **Toast state** — useToast → `useState<{message, type} | null>` → ~3 seconds

5.2 Session Management

Backend sessions: None. The **FastAPI backend** is fully **stateless**. No cookies, no **JWT tokens**, no server-side session store. Every **HTTP request** is independent.

Chat session memory: Managed entirely on the **frontend**. The **messages** array in **Chat.tsx** is the session. On every send, **messages.slice(1)** (all messages except the greeting) is serialized and sent to the **backend** as the **history** field. The **backend** reconstructs the **conversation context** from this array on each request.

Session persistence: Chat messages are persisted to **localStorage['chatMessages']** via **useEffect** on every state change. On page reload or navigation, messages are re-loaded from **localStorage**, so the chat session survives browser navigation within the same browser.

Session reset: Clicking "**Clear Chat"** replaces the messages array with a single fresh greeting message and updates localStorage. The next API call sends history: [], and the backend system prompt explicitly tells the model it has no memory of prior exchanges.

Report session: Generated reports are not kept in component state across navigation. If the user navigates away from /new-report, the generated report is lost from view (but already saved to localStorage history). The user can retrieve it from **/history.**

5.3 localStorage Persistence Strategy

```
Key: 'construction_reports'
Value: JSON.stringify(SavedReport[])
Updated: on every saveReport() or deleteReport() call
Read: on History page mount via getReports()

Key: 'chatMessages'
Value: JSON.stringify(Message[])
Updated: useEffect triggered on every messages state change in Chat.tsx
Read: useState initializer in Chat.tsx (lazy initial state)
```

Timestamps in **Message** objects are serialized as ISO strings and re-hydrated as new **Date(msg.timestamp)** on load to restore the Date type.

6. Caching

6.1 Frontend Caching

1. **Saved reports** — localStorage → permanent → cleared manually
2. **Chat messages** — localStorage → permanent → cleared via "Clear Chat"
3. **Suggested questions** — useState → component lifetime → reset on reload
4. **jsPDF library** — browser module cache → session → cleared via browser cache
5. **SPA assets (JS/CSS)** — browser HTTP cache → per cache headers

6.2 Backend Caching

1. **ConstructionRAG instance** — in-memory singleton → reused across requests
2. **Embedding model** — HuggingFace cache (disk) → loaded on startup
3. **ChromaDB index** — on-disk + in-memory → loaded once, reused
4. **Groq API responses** — not cached → always fresh

Note:

- ChromaDB HNSW index is loaded into memory at startup → no disk I/O per query
- No HTTP-level caching for API responses → all outputs are freshly generated

6.3 Embedding Model Loading

The **all-MiniLM-L6-v2** model (90MB) is **downloaded** from **HuggingFace Hub** on first run and **cached** at `/.cache/huggingface/hub/. On subsequent server starts, it **loads from the local cache** — no network request needed. This is handled **transparently** by the **sentence-transformers** library.

7. Non-Functional Requirements

7.1 Security Aspects

- **API Key Management**
  → Stored in `.env`, loaded via environment variables
  → Not exposed to frontend or responses
  → `.env` excluded from version control
  → Risk: key exposure
  → Mitigation: env-only storage + `.gitignore`

- **CORS Policy**
  → Restricted to known frontend origins (localhost)
  → Must update for production domain
  → Risk: unauthorized cross-origin access
  → Mitigation: explicit whitelist in backend

- **Input Handling**
  → Validated using Pydantic models
  → Type-checked, unexpected fields ignored
  → No SQL/shell execution (no injection surface)
  → LLM constrained via system prompt
  → Risk: prompt injection
  → Mitigation: strict prompt rules + low temperature + labeled RAG context

- **Data Privacy**
  → Stored in browser localStorage only
  → No external storage or database
  → Only prompt sent to Groq API
  → Risk: sensitive data sent to external API
  → Mitigation: avoid confidential input, future self-hosted LLM option

- **No Authentication**
  → API open on localhost
  → Suitable for dev/demo only
  → Risk: unauthorized access in networked setups
  → Mitigation: add API key or OAuth in production

7.2 Performance Aspects

LLM Inference Latency

1. **Detailed report** — ~3–8 seconds
2. **Brief report** — ~1–3 seconds
3. **Chat response** — ~1–4 seconds
4. **UX handling** — progress bar (report), typing indicator (chat)

Embedding + Vector Search

1. **ChromaDB search** — <50ms
2. **Embedding time** — ~100–200ms (first call), faster after load
3. **Optimization** — model stays in memory

Frontend Rendering Performance

- **react-markdown** renders markdown to React elements — for reports up to 700+ words this is imperceptible
- The report content area has **max-height: 700px** with overflow scroll to prevent layout reflow on large reports
- jsPDF is dynamically imported **(import('jspdf'))** — only loaded when the user clicks PDF download, keeping initial bundle size smaller
- Chat messages use **useRef + scrollIntoView** for smooth auto-scroll to latest message

localStorage Performance

LocalStorage operations are synchronous and block the main thread. For the data volumes in this application (tens of reports, hundreds of chat messages), this is not a concern. If the report count grows to thousands, `getReports()` (which parses the full array) could become slow — at that point, IndexedDB would be the appropriate upgrade.

Bundle Size

1. React + DOM — ~130KB
2. Router — ~25KB
3. Markdown libs — ~80KB
4. docx — ~200KB
5. file-saver — ~5KB
6. jsPDF — ~300KB (lazy)
7. Total initial — ~450KB

Scalability Considerations

1. **LLM inference** — Groq API → scale via multiple keys / self-hosted LLM
2. **Vector search** — local ChromaDB → scale via shared/cloud DB
3. **Report storage** — localStorage → scale via backend DB
4. **Backend** — single process → scale via multiple workers / containers

References

1. FastAPI Documentation — https://fastapi.tiangolo.com/
2. LangChain Documentation — https://python.langchain.com/docs/
3. Groq API Documentation — https://console.groq.com/docs/
4. ChromaDB Documentation — https://docs.trychroma.com/
5. HuggingFace sentence-transformers — https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
6. React Documentation — https://react.dev/
7. React Router v6 — https://reactrouter.com/en/main
8. react-markdown — https://github.com/remarkjs/react-markdown
9. remark-gfm — https://github.com/remarkjs/remark-gfm
10. docx npm package — https://docx.js.org/
11. jsPDF — https://github.com/parallax/jsPDF
12. file-saver — https://github.com/eligrey/FileSaver.js
13. Vite — https://vitejs.dev/
14. Pydantic v2 — https://docs.pydantic.dev/
15. HNSW Algorithm — https://arxiv.org/abs/1603.09320
16. RAG (Lewis et al. 2020) — https://arxiv.org/abs/2005.11401